

თბილისის თავისუფალი უნივერსიტეტი
მათემატიკის, კომპიუტერული მეცნიერების და ინჟინერიის
სკოლა (MACS[E])
კომპიუტერული მეცნიერებისა და მათემატიკის პროგრამა

არევაძე მათე
ჭიჭინაძე ქეთი
გელაშვილი გიორგი

საბაკალავრო პროექტი
„Peared“

ხელმძღვანელი: აბსანძე ლუკა

თბილისი
2026

ანოტაცია

ერთობლივი მუშაობა ერთსა და იმავე ტექსტურ ფაილზე დღეს ყოველდღიურობის ნაწილია - იქნება ეს წყვილში პროგრამირება, სემინარი, ლაბორატორიული სამუშაო თუ ტექნიკური გასაუბრება. ამის მიუხედავად, არსებული ინსტრუმენტების უმეტესობა ორ დაშვებას ეყრდნობა: დოკუმენტი უნდა „აიტვირთოს“ რომელიმე კომპანიის ცენტრალურ სერვერზე და ყველა მონაწილეს სტაბილური ინტერნეტი უნდა ჰქონდეს. ჩვენი გამოცდილებით, უნივერსიტეტის აუდიტორიაში ორივე დაშვება ხშირად ირღვევა: ინტერნეტის გაწყვეტისას ეკრანის გაზიარება ჩერდება და სემინარზე ყველა სტუდენტს ერთი ლეპტოპის გარშემო შეკრება უწევს.

ამ პრობლემის გადასაჭრელად შევექმენით Peered, კროსპლატფორმული desktop აპლიკაცია ერთობლივი ტექსტური რედაქტირებისთვის, რომელიც მუშაობს როგორც ლოკალურ ქსელში (LAN) ინტერნეტის გარეშე, ისე გლობალურ ქსელში. სესიის წამომწყები (ჰოსტი) საკუთარ კომპიუტერზე უშვებს მსუბუქ სარელეო სერვერს, რომელსაც აპლიკაცია ავტომატურად მართავს, ხოლო დანარჩენი მონაწილეები უბრალოდ ბმულით უერთდებიან. დოკუმენტი არასდროს ტოვებს მონაწილეთა კომპიუტერებს, ცენტრალური სერვისი, ანგარიშის შექმნა და გამოწერა საჭირო არ არის.

პარალელური რედაქტირებისას ცვლილებების უკონფლიქტოდ შერწყმას უზრუნველყოფს CRDT (Conflict-free Replicated Data Type) ალგორითმი, კონკრეტულად, YATA მიდგომა, რომელიც Yjs ბიბლიოთეკის საფუძველია. თითოეული კლიენტი დოკუმენტის საკუთარ ასლს ფლობს და ოპერაციები ისეა აგებული, რომ ნებისმიერი თანმიმდევრობით მიღების შემთხვევაში ყველა მონაწილე ერთსა და იმავე ტექსტამდე მიდის. სისტემა სამი დამოუკიდებელი კომპონენტისგან შედგება: Rust-ზე დაწერილი CRDT ბიბლიოთეკა, Tauri-ზე აგებული desktop კლიენტი (React/Monaco ფრონტენდით) და Go-ზე დაწერილი WebSocket სარელეო სერვერი. გლობალური წვდომისთვის აპლიკაცია ავტომატურად ხსნის Cloudflare Tunnel-ს, რაც პორტების გახსნისა და ქსელის კონფიგურაციის გარეშე აძლევს დაშორებულ მონაწილეებს შეერთების საშუალებას.

პროექტში განსაკუთრებული ყურადღება დაეთმო წარმადობასა და მეხსიერების ეფექტიანობას: პოზიციების ძებნის დასაჩქარებლად დოკუმენტს დავურთეთ აუგმენტირებული B+ ხე, რომელმაც საკვანძო ოპერაციები წრფივი დროიდან ლოგარითმულამდე დაიყვანა, ხოლო წაშლილი ტექსტის „საფლავის ქვების“ (tombstones) დასაგროვებლად შევიმუშავეთ ჰოსტზე ორიენტირებული ნაგვის შემგროვებელი, რომელიც

ჩვენი არქიტექტურის თავისებურებას, ჰოსტს, როგორც სიმართლის წყაროს, ეყრდნობა.

შედეგად მივიღეთ მომუშავე პროდუქტი: Linux-ზე, Windows-სა და macOS-ზე გაშვებადი რედაქტორი, რომლითაც რამდენიმე ადამიანს ერთდროულად შეუძლია ერთი და იმავე ფაილის რედაქტირება, უფლებების მართვა, ფაილების შენახვა/გახსნა და სესიის სრული სასიცოცხლო ციკლის კონტროლი.

Abstract

Working together on the same text file is part of everyday life today - whether it is pair programming, a seminar, a lab session or a technical interview. Despite this, most existing tools rest on two assumptions: the document must be uploaded to some company's central server, and every participant must have a stable internet connection. In our experience, both assumptions frequently break down in a university classroom: when the internet cuts out, screen sharing stops, and the whole seminar ends up crowding around a single laptop.

To solve this problem we built Peared, a cross-platform desktop application for collaborative text editing that works both on a local network (LAN) without internet access and over the global network. The session initiator (the host) runs a lightweight relay server on their own machine, managed automatically by the application, while the other participants simply join via a link. The document never leaves the participants' computers, no central service, no accounts and no subscriptions are required.

Conflict-free merging of concurrent edits is guaranteed by a CRDT (Conflict-free Replicated Data Type) algorithm, specifically the YATA approach that underlies the Yjs library. Every client owns its own replica of the document, and operations are constructed so that, regardless of the order in which they are received, all participants converge to the same text. The system consists of three independent components: a CRDT library written in Rust, a desktop client built on Tauri (with a React/Monaco frontend), and a WebSocket relay server written in Go. For global access the application automatically opens a Cloudflare Tunnel, which lets remote participants connect without port forwarding or any network configuration.

Special attention was paid to performance and memory efficiency: to speed up position lookups we augmented the document with a B+ tree index, which reduced key operations from linear to logarithmic time, and to reclaim the tombstones left behind by deleted text we designed a host-centric garbage collection scheme that takes advantage of a distinctive property of our architecture, the host acting as the source of truth.

The result is a working product: an editor that runs on Linux, Windows and macOS, with which several people can edit the same file simultaneously, manage write permissions, open and save files, and control the full life cycle of a collaboration session.

სარჩევი

ანოტაცია	ii
Abstract	iv
ცხრილების, დიაგრამების, ფორმულების და სურათების ჩამონათვალი	vi
შესავალი	1
ძირითადი ნაწილი	3
პროექტის თემასთან დაკავშირებული სფეროს კვლევა	3
პროექტის ტექნიკური მხარე	6
მიღებული შედეგი და შეფასება	22
დასკვნა	26
გამოყენებული ლიტერატურა	28
დანართები	29

ცხრილების, დიაგრამების, ფორმულების და სურათების ჩამონათვალი

დიაგრამები

- 1 სისტემის საერთო არქიტექტურა: ჰოსტის კომპიუტერზე
გაშვებული გეითვეი და მასთან დაკავშირებული კლიენტები . . . 6
- 2 კლიენტის (Tauri აპლიკაციის) შრეები და ქვესისტემები 8
- 3 CRDT დოკუმენტის სტრუქტურა: ბლოკების დაკავშირებული
სია (ქვემოთ, tombstone წითლადაა) და მასზე აგებული
პოზიციური B+ ინდექსი (ზემოთ, კვანძებში, ხილული
სიმბოლოების ჯამები) 11
- 4 ბლოკის ჩასმა ნაბიჯ-ნაბიჯ: ბლოკი ერთდროულად ემატება
დაკავშირებულ სიას და ხის ფოთოლს, შემდეგ კი ხილული
სიგრძის დელტა ფესვამდე ვრცელდება 12
- 5 ბლოკის წაშლა ნაბიჯ-ნაბიჯ: ბლოკი სიაში რჩება tombstone-
ად, ხილული სიგრძე ნულდება და უარყოფითი დელტა ხის
ჯამებს ასწორებს 13

სურათები

ცხრილები

- 1 პოზიციის ძეგნის დრო ინდექსამდე და ინდექსის შემდეგ 19

შესავალი

პროექტის თემის შერჩევა საკმაოდ პრაქტიკული დაკვირვებით დაიწყო. სწავლის პერიოდში არაერთხელ აღმოვჩენილვართ სიტუაციაში, როდესაც სემინარზე ან ლაბორატორიულ სამუშაოზე საჭირო იყო, რომ რამდენიმე სტუდენტს ერთდროულად ემუშავა ერთსა და იმავე კოდზე. სტანდარტული გამოსავალი, ეკრანის გაზიარება - უნივერსიტეტის ქსელში ხშირად უბრალოდ არ მუშაობდა: ინტერნეტი წყდებოდა, ვიდეო ნაკადი ჭედავდა და საბოლოოდ მთელი ჯგუფი ლექტორის ლეპტოპის გარშემო იკრიბებოდა, რომ სემინარში მონაწილეობა მიეღო. Google Docs-ის ტიპის ინსტრუმენტები კი, გარდა იმისა, რომ კოდის რედაქტირებისთვის მოუხერხებელია, სტაბილურ ინტერნეტსა და ცენტრალურ სერვერს საჭიროებს, ანუ ზუსტად იმას, რაც აუდიტორიაში ხშირად არ გვქონდა.

აქედან გაჩნდა კითხვა: შესაძლებელია თუ არა ისეთი ერთობლივი რედაქტორის შექმნა, რომელიც ცენტრალურ სერვისზე საერთოდ არ იქნება დამოკიდებული და იმუშავებს როგორც ლოკალურ ქსელში (თუნდაც ინტერნეტის გარეშე), ისე ინტერნეტით დაშორებულ მონაწილეებთანაც? თემის ამ მიმართულებით განვითარებაში დაგვეხმარა ჩვენი ხელმძღვანელი, ლუკა აბსანძე, რომელმაც პროექტის საწყის ეტაპზე რამდენიმე საინტერესო იდეა და მიმართულება შემოგვთავაზა, საიდანაც საბოლოო არჩევანი გავაკეთეთ.

პრობლემა, რომელსაც პროექტი ეხმიანება, შემდეგნაირად შეიძლება ჩამოყალიბდეს: რეალურ დროში ერთობლივი ტექსტური რედაქტირების არსებული გადაწყვეტილებები თითქმის ყველა შემთხვევაში მოითხოვს (1) დოკუმენტის მესამე მხარის სერვერზე შენახვას, (2) მუდმივ ინტერნეტკავშირს და (3) ხშირად ანგარიშების შექმნასა და კონკრეტულ ეკოსისტემაზე მიბმას. ეს სამივე მოთხოვნა პრობლემურია მაშინ, როდესაც სამუშაო ფაილი კონფიდენციალურია, ქსელი არასტაბილურია ან უბრალოდ გვინდა, რომ ერთი ოთახის ფარგლებში, ზედმეტი ინფრასტრუქტურის გარეშე ვითანამშრომლოთ.

პროექტის მიზანი იყო სრულფასოვანი, კროსპლატფორმული desktop აპლიკაციის შექმნა, რომელიც:

- საშუალებას აძლევს ერთ მომხმარებელს (ჰოსტს), წამოიწყოს სესია და დანარჩენებს, უბრალოდ ბმულით შეუერთდნენ მას;
- პარალელურ რედაქტირებას უკონფლიქტოდ წყვეტს CRDT ალგორითმით, ისე, რომ ყველა მონაწილე გარანტირებულად ერთსა და იმავე ტექსტს ხედავდეს;
- მუშაობს როგორც ლოკალურ ქსელში ინტერნეტის გარეშე,

ისე გლობალურად - Cloudflare Tunnel-ის მეშვეობით, ქსელის ყოველგვარი კონფიგურაციის გარეშე;

- ჰოსტს აძლევს სესიის სრულ კონტროლს: სტუმრების ჩაწერის უფლებების მართვას, სესიის დაწყება/დასრულებას და ფაილების ლოკალურად შენახვას.

პროექტის ფარგლები შეგნებულად შემოვსაზღვრეთ: სესია ერთ დოკუმენტზე მუშაობს, დოკუმენტი უბრალო ტექსტია (კოდი, კონფიგურაცია, ჩანაწერები) და არა ფორმატირებული rich text, ხოლო სესია ეფემერულია, ის ჰოსტის აპლიკაციასთან ერთად იბადება და კვდება, თუმცა ნამუშევრის ფაილად შენახვა ნებისმიერ მომენტშია შესაძლებელი.

მეთოდოლოგიის მხრივ სისტემა სამ დამოუკიდებლად აწყობად კომპონენტად დაყავით: CRDT ბიბლიოთეკა Rust-ზე (crdt-core), desktop კლიენტი Tauri ფრეიმვორკზე (Rust ბექენდი + React/TypeScript ფრონტენდი Monaco რედაქტორით) და მსუბუქი სარელეო სერვერი Go-ზე (gateway). განვითარების პროცესი GitHub-ზე მიმდინარეობდა: ფუნქციონალი იშლებოდა issue-ებად, თითოეული ცვლილება pull request-ით და კოდის განხილვით (code review) ერთიანდებოდა, ხოლო CI ავტომატურად უშვებდა ფორმატირების, ლინტერების, ტესტებისა და უსაფრთხოების შემოწმებებს სამივე კომპონენტზე.

პოტენციური მომხმარებლები, პირველ რიგში, სტუდენტები და ლექტორები არიან - სემინარები, ლაბორატორიული სამუშაოები, ჯგუფური პროექტები, თუმცა იგივე სცენარები ბუნებრივად ვრცელდება წყვილში პროგრამირებაზე, ტექნიკურ გასაუბრებებზე, მენტორინგსა და ნებისმიერ სიტუაციაზე, სადაც რამდენიმე ადამიანს ერთი ტექსტის ერთდროული რედაქტირება სჭირდება ცენტრალური სერვისისთვის მონაცემების გადაცემის გარეშე.

დაბოლოს, პროექტს აქვს რამდენიმე მოსალოდნელი შეზღუდვა. სესია ჰოსტის კომპიუტერზეა მიბმული: თუ ჰოსტი გაითიშა, სესია სრულდება (თუმცა თითოეულ მონაწილეს დოკუმენტის სრული ასლი რჩება). გლობალური წვდომა დამოკიდებულია Cloudflare-ის უფასო trycloudflare.com სერვისის ხელმისაწვდომობაზე - მისი გაუმართაობის შემთხვევაში აპლიკაცია ლოკალური ქსელის რეჟიმში აგრძელებს მუშაობას. ასევე, რადგან წაშლილი ტექსტის კვალი CRDT ალგორითმის კორექტულობისთვის გარკვეული ხნით უნდა შენარჩუნდეს, მეხსიერების მოხმარება დამოკიდებულია რედაქტირების ისტორიაზე, ამ პრობლემის შესარბილებლად სპეციალური ნაგვის შემგროვებელი მექანიზმი შევიმუშავეთ, რომელსაც შემდეგ თავებში დეტალურად აღვწერთ.

ძირითადი ნაწილი

პროექტის თემასთან დაკავშირებული სფეროს კვლევა

ერთობლივი რედაქტირება და არსებული ტექნოლოგიები

რეალურ დროში ერთობლივი რედაქტირება დღეს ყველასთვის ნაცნობი ფუნქციაა - Google Docs-მა ის მასობრივ პროდუქტად აქცია, ხოლო პროგრამისტებისთვის არსებობს სპეციალიზებული ინსტრუმენტები: VS Code Live Share, JetBrains Code With Me, ბრაუზერული Replit და CodeSandbox, ღია კოდის Etherpad და სხვა. თუ ამ ინსტრუმენტებს დავაკვირდებით, თითქმის ყველა მათგანი ერთსა და იმავე არქიტექტურულ ნიმუშს მიჰყვება: არსებობს ცენტრალური სერვერი, რომელიც დოკუმენტის „კანონიკურ“ ვერსიას ფლობს, ხოლო კლიენტები ამ სერვერს უკავშირდებიან. ეს მიდგომა კარგად მუშაობს კომერციული SaaS პროდუქტისთვის, მაგრამ მასთან ერთად მოდის დამოკიდებულებები: მუდმივი ინტერნეტი, მესამე მხარის ინფრასტრუქტურა და დოკუმენტის მონაცემების სხვის სერვერზე გატანა.

ტექნიკური თვალსაზრისით, პარალელური რედაქტირების შერწყმის პრობლემას ორი ძირითადი სკოლა პასუხობს. პირველი და ისტორიულად უფრო ადრეული არის **Operational Transformation (OT)** [3]: როდესაც ორი მომხმარებელი ერთდროულად ასწორებს ტექსტს, მათი ოპერაციები (ჩასმა/წაშლა პოზიციებით) ერთმანეთის მიმართ „ტრანსფორმირდება“ ისე, რომ განსხვავებული თანმიმდევრობით შესრულების შემდეგაც ერთი და იგივე შედეგი მივიღოთ. OT გამოიყენება Google Docs-სა და Etherpad-ში, თუმცა მას ორი ცნობილი სისუსტე აქვს: ტრანსფორმაციის ფუნქციების კორექტულად დაწერა ძალიან რთულია (ამ სფეროში არაერთი გამოქვეყნებული ალგორითმი მოგვიანებით მცდარი აღმოჩნდა), და პრაქტიკული OT სისტემები თითქმის ყოველთვის ცენტრალურ სერვერს საჭიროებენ, რომელიც ოპერაციებს ერთიან თანმიმდევრობას მიანიჭებს.

მეორე მიდგომაა **CRDT, Conflict-free Replicated Data Type** [2]. აქ იდეა შებრუნებულია: იმის ნაცვლად, რომ ოპერაციები ერთმანეთს მოვარგოთ, თავად მონაცემთა სტრუქტურა ისეა აგებული, რომ ოპერაციები კომუტაციური იყოს, ანუ მიღების თანმიმდევრობას შედეგი არ შეეცვალოს. თითოეული ჩასმული სიმბოლო (ან სიმბოლოთა ბლოკი) იღებს გლობალურად უნიკალურ იდენტიფიკატორს და პოზიციის ნაცვლად სწორედ ამ იდენტიფიკატორებით აღიწერება „სად“ ჩაჯდა ტექსტი. ამის ფასი ისაა, რომ წაშლილი ელემენტების კვალი, ე.წ. tombstone-ები, გარკვეული ხნით უნდა შევინახოთ, სამაგიეროდ ცენტრალური

კოორდინატორი საერთოდ აღარ არის საჭირო: CRDT ბუნებრივად ერგება peer-to-peer და დეცენტრალიზებულ არქიტექტურებს.

ტექსტისთვის განკუთვნილი არაერთი CRDT ალგორითმი არსებობს (WOOT, RGA, Logoot/LSEQ და სხვა), მაგრამ პრაქტიკაში ყველაზე წარმატებული აღმოჩნდა **YATA, Yet Another Transformation Approach** [1], რომელიც Yjs ბიბლიოთეკის [4] საფუძველია. Yjs დღეს ალბათ ყველაზე ფართოდ გამოყენებადი ღია კოდის ერთობლივი რედაქტირების ძრავაა და სწორედ მისმა პრაქტიკულმა წარმატებამ დაგვარწმუნა, რომ YATA სწორი არჩევანი იყო. YATA-ში ყოველი ჩასმა იმასსოვრებს არა მხოლოდ საკუთარ იდენტიფიკატორს, არამედ „წარმოშობის“ მეზობლებსაც (origin left/right), თუ რომელ ორ ელემენტს შორის დაიბადა ის. როდესაც ორი მონაწილე ერთსა და იმავე ადგილას ერთდროულად ჩასვამს ტექსტს, კონფლიქტი დეტერმინისტული წესით წყდება და, რაც მნიშვნელოვანია, პარალელურად აკრეფილი სიტყვები ერთმანეთში არ „აირევა“, ალგორითმი გამორიცხავს ე.წ. interleaving-ის პრობლემას, რომლითაც ზოგიერთი სხვა მიდგომა (მაგ. პოზიციური იდენტიფიკატორებზე დაფუძნებული Logoot) იტანჯება.

რატომ CRDT და რატომ YATA

ჩვენი არჩევანი სამ არგუმენტს ეყრდნობოდა. ჯერ ერთი, პროექტის მთავარი მოტივაცია სწორედ ცენტრალურ სერვისზე დამოკიდებულების მოხსნა იყო, OT-ის სერვერცენტრული ბუნება ამ მიზანს პირდაპირ ეწინააღმდეგება, CRDT კი საშუალებას გვაძლევს, სერვერული კომპონენტი უბრალო გადამგზავნ „მილად“ ვაქციოთ, რომელსაც დოკუმენტის შინაარსი საერთოდ არ ესმის. მეორე, CRDT-ის კორექტულობის მსჯელობა ლოკალურია: საკმარისია დავამტკიცოთ, რომ ოპერაციების ინტეგრაცია კომუტაციურია, და ქსელის ნებისმიერი გაჭიანურება თუ თანმიმდევრობის დარღვევა უსაფრთხო ხდება. მესამე, YATA-ს აქვს რეალურ პროდუქტში (Yjs) წლების განმავლობაში გამოცდილი დიზაინი, საიდანაც ბევრი პრაქტიკული ხერხი გადმოვიღეთ: სიმბოლოების ნაცვლად ბლოკებით (რამდენიმე სიმბოლოს გაბმული მონაკვეთებით) ოპერირება და ბლოკის გაყოფა (splitting) შუაში ჩასმისას, წაშლების კომპაქტური, დიაპაზონებად შენახვა და ა.შ. დამატებით შთაგონებას ვიღებდით diamond-types პროექტიდანაც [5], რომელმაც გვიჩვენა, რომ CRDT დოკუმენტს პოზიციური ინდექსით მნიშვნელოვნად აჩქარება შეუძლია.

რატომ Cloudflare Tunnel

დეცენტრალიზებული მიდგომის ერთი მოუხერხებელი დეტალი ისაა, რომ ჰოსტის კომპიუტერი ინტერნეტიდან პირდაპირ მიუწვდომელია: სახლისა თუ უნივერსიტეტის ქსელებში NAT და firewall გარე შეერთებებს ბლოკავს, ხოლო პორტების ხელით გახსნა (port forwarding) რიგითი მომხმარებლისთვის შეუძლებელი მოთხოვნაა. ამ პრობლემას ვჭრით Cloudflare Tunnel-ით [8]: ჰოსტის აპლიკაცია cloudflared პროგრამას უშვებს, რომელიც ჰოსტიდან Cloudflare-ის ქსელისკენ გამავალ კავშირს ამყარებს და სანაცვლოდ დროებით საჯარო მისამართს (*.trycloudflare.com) იღებს. გამავალი კავშირი NAT-ს უპრობლემოდ გადის, მომხმარებელს არაფრის კონფიგურაცია არ სჭირდება, კავშირი TLS-ითაა დაცული და სერვისი უფასოა. თუ გვირავის გახსნა ვერ მოხერხდა, აპლიკაცია ამაზე არ ვარდება, სესია ლოკალური ქსელის ბმულით გრძელდება.

პროექტის ბუნება, მოთხოვნები და კომერციალიზაცია

პროექტი არსებულ სფეროში არსებული გამოწვევის, ცენტრალიზებულ ინფრასტრუქტურაზე დამოკიდებულების, ალტერნატიულ გადაწყვეტას გვთავაზობს. ჩვენ არ გამოგვიგონია ახალი თანხმობის ალგორითმი; სიახლე კომბინაციაშია: local-first desktop აპლიკაცია, რომელიც სესიის ინფრასტრუქტურას (სარელეო სერვერს და საჯარო გვირავს) მომხმარებლისვე კომპიუტერზე, ავტომატურად და შეუმჩნევლად მართავს. ცალკე აღნიშვნის ღირსია ჩვენი ნაგვის შემგროვებელი სისტემა, რომელიც, ჩვენი ცოდნით, CRDT იმპლემენტაციებში გავრცელებული მიდგომებისგან განსხვავებულ, ჰოსტ-ავტორიტეტულ სქემას იყენებს (დეტალურად - ოპტიმიზაციების თავში).

ტექნიკური მოთხოვნების მხრივ პროექტი მთლიანად პროგრამულია და აპარატურულ კომპონენტს არ საჭიროებს: განვითარებისთვის საკმარისია Rust, Go და Node.js ინსტრუმენტები, ხოლო საბოლოო მომხმარებლისთვის, ჩვეულებრივი კომპიუტერი Linux-ით, Windows-ით ან macOS-ით. კომერციალიზაციის კუთხით პროექტს რამდენიმე რეალისტური მიმართულება აქვს: (1) საგანმანათლებლო ლიცენზირება - ინსტრუმენტი, რომელიც სუსტი ინფრასტრუქტურის მქონე აუდიტორიებში მუშაობს, სკოლებისა და უნივერსიტეტებისთვის ბუნებრივი პროდუქტია; (2) თვითჰოსტინგზე ორიენტირებული საწარმოო ვერსია იმ კომპანიებისთვის, რომლებსაც კოდის მესამე მხარის სერვისში გატანა შიდა პოლიტიკით ეკრძალებათ; (3) ტექნიკური გასაუბრებების ნიშური ბაზარი, სადაც სესიის ეფემერულობა და მონაცემების ლოკალურობა უპირატესობაა.

ამასთან, გულწრფელად უნდა ითქვას, რომ პროექტის პირველადი მიზანი საგანმანათლებლო იყო და კომერციალიზაცია მომავლის პერსპექტივად რჩება.

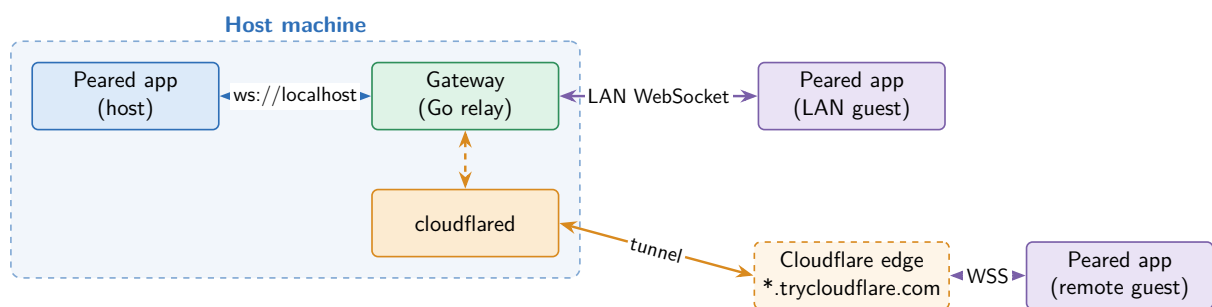
პროექტის ტექნიკური მხარე

სისტემის საერთო არქიტექტურა

Peared-ის სისტემა სამი გაშვებადი (runtime) კომპონენტისგან შედგება:

1. **desktop აპლიკაცია** (tauri-app), მას ყველა მონაწილე უშვებს. სწორედ აქ ცხოვრობს დოკუმენტის CRDT ასლი, Monaco რედაქტორი და სესიის მართვის მთელი ლოგიკა.
2. **გეითვეი** (gateway), Go-ზე დაწერილი WebSocket სარელო სერვერი, რომელსაც მხოლოდ ჰოსტის აპლიკაცია უშვებს დამხმარე (sidecar) პროცესად სესიის დაწყებისას და კლავს სესიის დასრულებისას. ის მდგომარეობას თითქმის არ ინახავს, უბრალოდ გადააგზავნის ბაიტებს ოთახის წევრებს შორის.
3. **Cloudflared გვირაბი**, არასავალდებულო დამხმარე პროცესი, რომელიც ჰოსტის გეითვეის დროებით საჯარო *.trycloudflare.com მისამართზე აქვეყნებს, რათა ლოკალური ქსელის გარეთ მყოფმა სტუმრებმაც შეძლონ შეერთება.

ქსელური ტოპოლოგია hub-and-spoke მოდელს მიჰყვება: ყველა მონაწილე (თავად ჰოსტის კლიენტიც კი) ერთსა და იმავე გეითვეის უკავშირდება WebSocket-ით, ხოლო გეითვეი მიღებულ ყოველ ფრეიმს დანარჩენებს გადაუგზავნის. ამით ვიღებთ peer-to-peer სისტემის თვისებებს (დოკუმენტი მხოლოდ მონაწილეებთანაა, ცენტრალური სერვისი არ არსებობს) სრული mesh-ქსელის სირთულის გარეშე - თითოეულ კლიენტს მხოლოდ ერთი კავშირი სჭირდება.



დიაგრამა 1. სისტემის საერთო არქიტექტურა: ჰოსტის კომპიუტერზე გაშვებული გეითვეი და მასთან დაკავშირებული კლიენტები

სესიის სასიცოცხლო ციკლი. როდესაც მომხმარებელი სესიის წამოწყებას (hosting) ირჩევს, აპლიკაცია თანმიმდევრულად ასრულებს შემდეგ ნაბიჯებს: (1) უშვებს გეითვეის პროცესს და მისგან სტანდარტულ გამოსავალზე (stdout) დაბეჭდილი JSON შეტყობინებით იგებს, რომელ პორტზე დაიწყო მუშაობა; (2) საუკეთესო ძალისხმევით პრინციპით უშვებს cloudflared-ს და ელოდება საჯარო ბმულს; (3) HTTP მოთხოვნით გეითვეისგან ქმნის „ოთახს“ (room) და იღებს მის იდენტიფიკატორს; (4) თავად უერთდება ოთახს WebSocket-ით და მაშინვე აგზავნის დოკუმენტის სნეფშოტს, რათა მოგვიანებით შემოსული სტუმრები სწრაფად „დაეწიონ“ მიმდინარე მდგომარეობას. ამის შემდეგ ჰოსტი ორ ბმულს იღებს, ლოკალური ქსელისა და საჯაროს, და ორივეს ერთი ღილაკით გააზიარებს. სტუმრის მხარეს იგივე პროცესი ბევრად მარტივია: ბმულის ჩასმა, WebSocket კავშირი, სნეფშოტის მიღება და მუშაობის დაწყება.

სესიაში უფლებები ასიმეტრიულია: ჰოსტი ყოველთვის სრულუფლებიანია, ხოლო სტუმრები, ნაგულისხმევად, მხოლოდ კითხვის რეჟიმში ერთვებიან (ჰოსტს შეუძლია სესიის დაწყებისას აირჩიოს, რომ სტუმრებს ჩაწერის უფლება ავტომატურად მიეცეთ). სესიის მიმდინარეობისას ჰოსტი თითოეულ სტუმარს ინდივიდუალურად ანიჭებს ან ართმევს ჩაწერის უფლებას, ამ მექანიზმის ტექნიკურ დეტალებს ქვემოთ განვიხილავთ.

Tauri ფრეიმვორკი კონცეპტუალურად

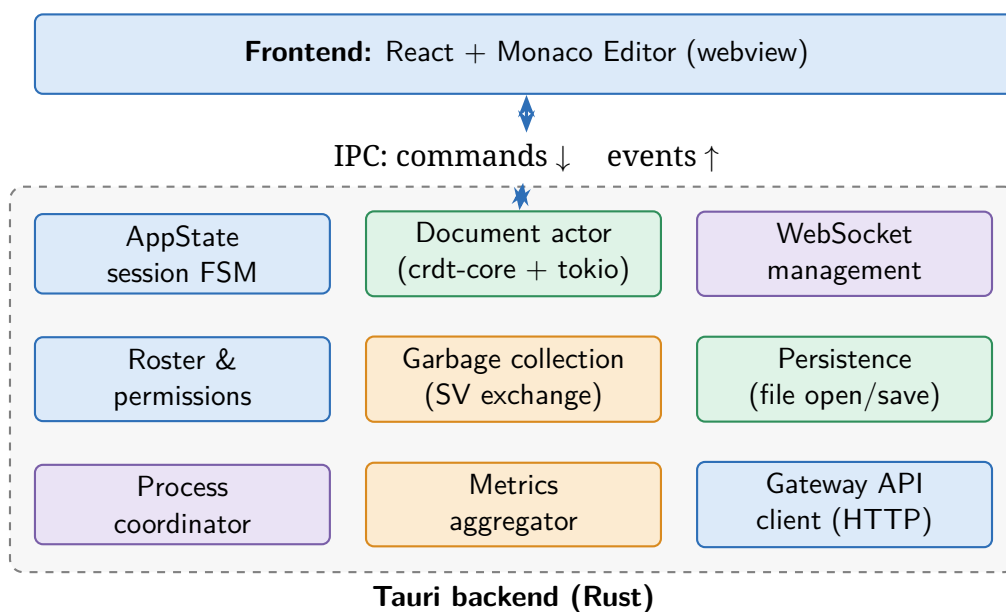
desktop აპლიკაცია აგებულია Tauri 2 ფრეიმვორკზე [6]. Tauri-ის იდეა მარტივია: აპლიკაციის ინტერფეისი იწერება ჩვეულებრივი ვებ-ტექნოლოგიებით (ჩვენს შემთხვევაში React + TypeScript) და ეშვება ოპერაციული სისტემის ჩაშენებულ webview-ში, ხოლო „ნატიური“ ლოგიკა, ფაილებთან წვდომა, ქსელი, პროცესების მართვა, იწერება Rust-ზე და ცალკე პროცესის ბირთვში ცხოვრობს. ორ სამყაროს შორის კომუნიკაცია ხდება IPC-ით: ფრონტენდი იძახებს ბექენდის „ბრძანებებს“ (commands), ბექენდი კი ფრონტენდს „ივენტებს“ (events) უგზავნის. Electron-თან შედარებით Tauri-ის ბინარები გაცილებით მცირეა (ის Chromium-ს არ აპაკეტებს) და ბექენდის ენად Rust გვქონდა, რაც ჩვენთვის გადამწყვეტი იყო - CRDT ბიბლიოთეკა სწორედ Rust-ზეა დაწერილი და აპლიკაციაში პირდაპირ, დანამატების გარეშე ჩაშენდა.

Tauri-ის კიდევ ერთი ფუნქცია, რომელსაც აქტიურად ვიყენებთ, არის sidecar ბინარები: გეითვეი და cloudflared აპლიკაციასთან ერთად იდება და აპლიკაცია მათ სასიცოცხლო ციკლს სრულად აკონტროლებს - მომხმარებელი ვერც კი ამჩნევს, რომ მის კომპიუტერზე ცალკე სერვერი

გაეშვა.

ფრონტენდი და Monaco, როგორც ჩონჩხი

ინტერფეისის ბირთვად Monaco Editor [7] ავირჩიეთ, იგივე რედაქტორის კომპონენტი, რომელზეც VS Code დგას. ეს არჩევანი ბევრ რამეს გვაძლევს „უფასოდ“: სინტაქსის გაფერადება, მრავალკურსორიანი რედაქტირება, undo/redo, ძებნა და კლავიატურის ნაცნობი მალსახმობები. ჩვენი ფრონტენდი Monaco-ს გარშემოა შემოწერილი: React მართავს აპლიკაციის „გარსს“ (გვერდითი პანელი, სესიის სტატუსი, მონაწილეთა სია, ფაილების მენიუ), ხოლო Monaco - თავად ტექსტს. ლოკალური ცვლილებები Monaco-დან ბექენდისკენ IPC ბრძანებებით მიდის, დაშორებული მონაწილეების ცვლილებები კი ბექენდიდან ივენთებით ბრუნდება და რედაქტორში ისე ჩაესმება, რომ ხელახლა არ გავრცელდეს. ამ ორმხრივი ნაკადის დეტალებს ფრონტენდის თავში აღვწერთ.



დიაგრამა 2. კლიენტის (Tauri აპლიკაციის) შრეები და ქვესისტემები

CRDT ბიბლიოთეკა (crdt-core)

სისტემის გულია crdt-core, Rust-ზე დაწერილი დამოუკიდებელი ბიბლიოთეკა, რომელიც YATA-ს სტილის ტექსტურ CRDT-ს ახორციელებს. ის არაფერს იცის არც ქსელის, არც ინტერფეისის შესახებ: მისი პასუხისმგებლობაა დოკუმენტის მდგომარეობა, ოპერაციების ინტეგრაცია და ბინარული (wire) ფორმატი.

ბლოკები და დაკავშირებული სია. დოკუმენტი წარმოდგენილია ბლოკების ორმაგად დაკავშირებულ სიად. ბლოკი არის ერთი ავტორის მიერ ზედიზედ აკრეფილი სიმბოლოების მონაკვეთი, რომელსაც გააჩნია:

- უნიკალური იდენტიფიკატორი $BlockId = (client_id, clock)$ - ავტორის ნომერი და მისი ლოკალური მთვლელი (ლამპორტის საათის ანალოგი). გრძელი ბლოკი იდენტიფიკატორთა მთელ დიაპაზონს ფარავს: n სიმბოლოიანი ბლოკი (c, k) -დან $(c, k+n-1)$ -მდე;
- წარმოშობის მაჩვენებლები $origin_left$ და $origin_right$, რომელიც ორი ელემენტის შუაში დაიბადა ბლოკი ჩასმის მომენტში;
- შიგთავსი და წაშლის ნიშანი ($deleted$).

მთელი ტექსტის სიმბოლო-სიმბოლო შენახვის ნაცვლად ბლოკებით ოპერირება (მიდგომა, რომელიც Yjs-იდან გადმოვიღეთ [4]) მეხსიერებასა და ქსელის ტრაფიკს მკვეთრად ამცირებს: ჩვეულებრივი ბეჭდვისას მომდევნო სიმბოლო წინა ბლოკს ეწებება და ერთ ოპერაციად რჩება. ფაილის გახსნისას ტექსტი 64-სიმბოლოიან ბლოკებად იჭრება ($from_text_chunked$), რაც კარგი შუალედია „ბევრ პატარა ობიექტსა“ და „ერთ გიგანტურ სტრიქონს“ შორის.

ბლოკის გაყოფა (splitting). რა ხდება, როდესაც ვინმე ბლოკის შუაში ჩასვამს ტექსტს ან შუიდან წაშლის? ბლოკი ორად იყოფა: მარცხენა ნახევარი ინარჩუნებს საწყის იდენტიფიკატორს, მარჯვენა კი იღებს იმავე დიაპაზონის გაგრძელებას ($(c, k+offset)$). რადგან იდენტიფიკატორები დიაპაზონულია, გაყოფა „უფასოა“, არცერთი არსებული მიმართვა არ ტყდება: ნებისმიერი $BlockId$ კვლავ ცალსახად პოულობს თავის ბლოკს, უბრალოდ ახლა შესაძლოა ბლოკის შუაგულში აღმოჩნდეს. ეს ინვარიანტი (ე.წ. $decomposition\ invariance$) კრიტიკულია: დაშორებული მონაწილის ოპერაცია შეიძლება ისეთ ბლოკს მიემართებოდეს, რომელიც ჩვენთან უკვე სამ ნაწილად არის გაყოფილი.

ინტეგრაცია და კონფლიქტების გადაწყვეტა. დაშორებული ბლოკის მიღებისას ალგორითმმა უნდა იპოვოს მისი ზუსტი ადგილი სიაში. თუ კონფლიქტი არ არის, ბლოკი უბრალოდ თავის $origin$ -ებს შორის ჯდება. თუ ორმა მონაწილემ ერთსა და იმავე ადგილას ერთდროულად ჩასვა ტექსტი (ორივე ბლოკს ერთი და იგივე $origin$ -ები აქვს), YATA წესრიგს დეტერმინისტული წესით ადგენს, კონფლიქტური ბლოკები ისე ლაგდება, რომ (1) ყველა რეპლიკაზე ერთი და იგივე თანმიმდევრობა მიიღება და (2)

ერთი ავტორის მიმდევრობითი ჩანაწერები ერთად რჩება, ერთმანეთში არ ეწნება. Rust-ის ტიპთა სისტემა აქ ძალიან დაგვეხმარა: ინტეგრაციის ლოგიკა მკაცრად გამიჯნულ მოდულებშია (`document/integrate.rs`) და ყოველი უცნაური შემთხვევა ცალკე ტესტირება დაფარული.

წაშლა, DeleteSet და მდგომარეობის ვექტორი. CRDT-ში წაშლა განსხვავებულად მუშაობს: ბლოკის ფიზიკურად ამოღება არ შეიძლება, რადგან სხვა მონაწილის ჯერ მოუსვლელი ოპერაცია შესაძლოა სწორედ მას ეყრდნობოდეს `origin`-ად. ამიტომ წაშლა მხოლოდ მონიშვნაა: ბლოკი რჩება სიაში, როგორც „საფლავის ქვა“ (tombstone), ტექსტში კი აღარ ჩანს. წაშლები კომპაქტურად ინახება DeleteSet სტრუქტურაში, თითოეული კლიენტისთვის წაშლილი `clock`-დიაპაზონების სიად. ეს წარმოდგენა ორმაგად გამოსადეგია: ქსელით გადასაცემად მცირეა (ათასი ზედიზედ წაშლილი სიმბოლო ერთი დიაპაზონია) და იდემპოტენტურია, ერთი და იმავე DeleteSet-ის ორჯერ მიღება არაფერს აფუჭებს.

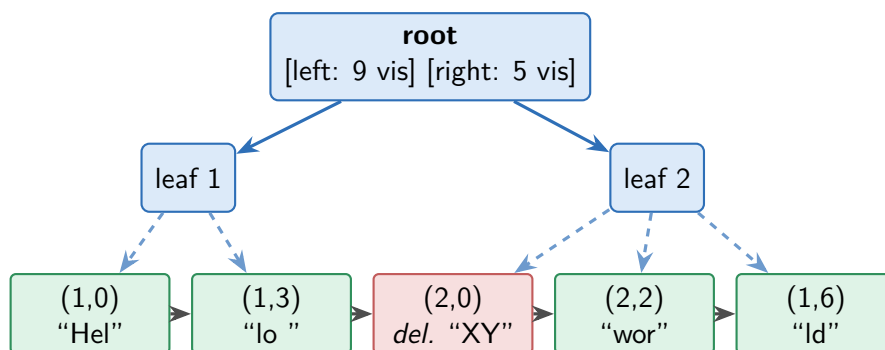
დოკუმენტის „სად ვარ“ მდგომარეობას აღწერს მდგომარეობის ვექტორი (state vector): თითო კლიენტზე, მისგან ნანახი ოპერაციების მაქსიმალური `clock`. ორი რეპლიკის ვექტორების შედარებით ზუსტად ჩანს, რომელ ოპერაციებს აკლია რომელიმე მხარეს. ჩვენ ვექტორებს ორ რამეში ვიყენებთ: გვიან შემოსული სტუმრის სინქრონიზაციასა და ნაგვის შემგროვებელში (იხ. ოპტიმიზაციების თავი), სადაც ის განსაზღვრავს, რომელი წაშლა აქვს ყველას გარანტირებულად ნანახი.

სნეფშოტები და wire პროტოკოლი. დოკუმენტის სრული მდგომარეობა, ბლოკები, DeleteSet, მდგომარეობის ვექტორი - სერიალიზდება Snapshot სტრუქტურად (ბინარულად, `bitcode` ბიბლიოთეკით). ქსელში ყოველ ფრეიმს ერთბაიტიანი პრეფიქსი აქვს, რომელიც ტიპს განსაზღვრავს: ოპერაცია (`0x00`), სნეფშოტი (`0x01`), საკონტროლო ფრეიმი (`0x02`), ნაგვის შემგროვებლის `commit` (`0x04`), წევრობის ცვლილება (`0x05`), მდგომარეობის ვექტორის რეპორტი (`0x06`), უფლების ცვლილება (`0x07`) და მონაწილის ინფორმაცია (`0x08`). რადგან ამ ფორმატს ორი ენა კითხულობს (Rust კლიენტებში, Go გეითვეიში), ორივე მხარეს გვაქვს ე.წ. `protocol drift` ტესტები, ბაიტ-ბაიტ დაფიქსირებული ნიმუშები, რომლებიც მაშინვე გატყდება, თუ ვინმემ ფორმატი ცალმხრივად შეცვალა.

პოზიციური B+ ინდექსი. CRDT ოპერაციები იდენტიფიკატორებზე მუშაობს, რედაქტორი კი, სიმბოლოს პოზიციებზე („ჩასვი 42-ე პოზიციაზე“).

ამ ორ სამყაროს შორის გადაყვანა დაკავშირებული სიის გავლით $O(n)$ ოპერაციაა და ყოველ კლავიშზე მთელი დოკუმენტის ჩამოვლა დიდ ფაილებზე შესამჩნევად ანელებდა სისტემას. ამის გადასაჭრელად დოკუმენტს დავურთეთ აუგმენტირებული B+ ხე: ბლოკები ხის ფოთლებში დოკუმენტისავე თანმიმდევრობით აწყვია, ხოლო ყოველი შიდა კვანძი ინახავს ქვეხის ხილული სიმბოლოების ჯამურ სიგრძეს (`visible_len`; წაშლილი ბლოკის ხილული სიგრძე ნულია). პოზიციით ძებნისას ხეზე დაშვებისას თითო დონეზე ვაკლებთ მარცხენა შვილების ჯამებს, $O(\log n)$; პირიქით, ბლოკიდან პოზიციის გამოთვლისას ფოთლიდან ზემოთ ავდივართ და მარცხენა ძმების ჯამებს ვაგროვებთ. ჩასმა, გაყოფა და წაშლა ხეში სარკისებურად აისახება და დელტა ფესვამდე „ბუშტდება“. დიზაინის შთაგონება `diamond-types` პროექტიდან [5] მოვიდა.

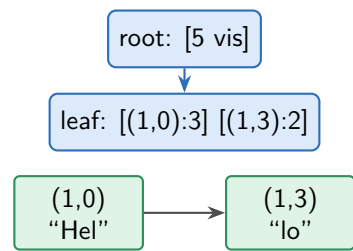
მნიშვნელოვანი დეტალი: ხე მეორეული, წარმოებული სტრუქტურაა - ჭეშმარიტების წყარო ყოველთვის დაკავშირებული სია რჩება. იმისთვის, რომ ისინი არასდროს დაშორდნენ, `debug` რეჟიმში ყოველი მუტაციის შემდეგ ეშვება „ორაკული“: ფუნქცია, რომელიც მთელ სიას ხელით ჩამოუვლის და ამოწმებს, რომ ხის პასუხები ზუსტად ემთხვევა, შეუსაბამობისას პროგრამა მაშინვე ვარდება `panic`-ით. ამან განვითარების პროცესში არაერთი ფაქიზი შეცდომა დაიჭირა, ხოლო `release` ბილდში ეს შემოწმება საერთოდ არ კომპილირდება და წარმადობას არ აზარალებს. ხის განშტოების ფაქტორებიც განზრახ განსხვავებულია: `debug`-ში მცირეა (4), რომ ტესტებში გაყოფის ლოგიკა ხშირად ვარჯიშობდეს, `release`-ში კი ფართო (64/32), ნაკლები დონე და უკეთესი კემ-ლოკალურობა.



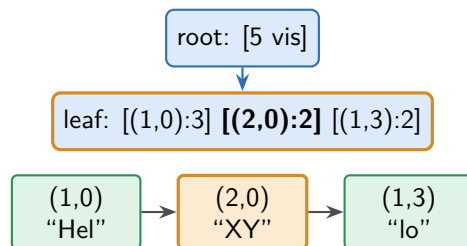
დიაგრამა 3. CRDT დოკუმენტის სტრუქტურა: ბლოკების დაკავშირებული სია (ქვემოთ, tombstone წითლადაა) და მასზე აგებული პოზიციური B+ ინდექსი (ზემოთ, კვანძებში, ხილული სიმბოლოების ჯამები)

ჩასმისა და წაშლის დინამიკას ორი კადრ-კადრ დაშლილი დიაგრამა აჩვენებს (დიაგრამები 4 და 5): პირველი, თუ როგორ ჯდება ახალი ბლოკი ერთდროულად სიაშიც და ხის ფოთოლშიც და როგორ „ბუბლდება“ ხილული სიგრძის დელტა ფესვამდე; მეორე, თუ როგორ იქცევა ბლოკი tombstone-ად და როგორ სწორდება ჯამები მთელ შტოზე.

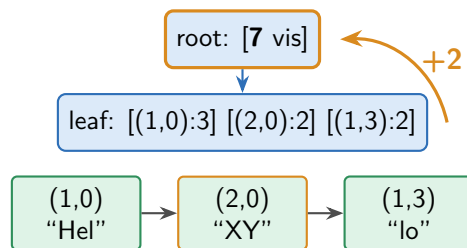
ნაბიჯი 1, საწყისი მდგომარეობა



ნაბიჯი 2, ახალი ბლოკი ჯდება სიაშიც და ფოთოლშიც

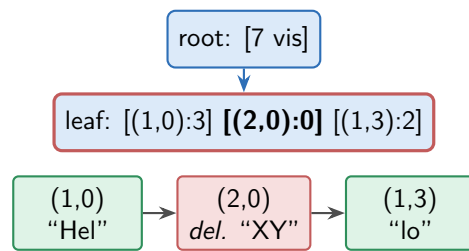


ნაბიჯი 3, დელტა (+2) ბუბლდება ფესვამდე

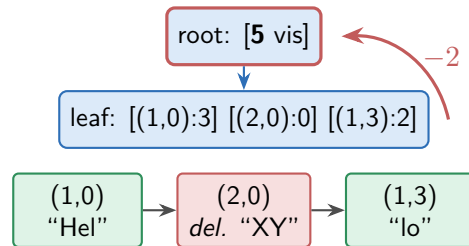


დიაგრამა 4. ბლოკის ჩასმა ნაბიჯ-ნაბიჯ: ბლოკი ერთდროულად ემატება დაკავშირებულ სიას და ხის ფოთოლს, შემდეგ კი ხილული სიგრძის დელტა ფესვამდე ვრცელდება

ნაბიჯი 1, ბლოკი ინიშნება წაშლილად (tombstone)



ნაბიჯი 2, დელტა (-2) ბუბლდება ფესვამდე



დიაგრამა 5. ბლოკის წაშლა ნაბიჯ-ნაბიჯ: ბლოკი სიაში რჩება tombstone-ად, ხილული სიგრძე ნულდება და უარყოფითი დელტა ხის ჯამებს ასწორებს

Tauri აპლიკაცია (desktop კლიენტი)

Tauri აპლიკაციის Rust ბექენდი ის ადგილია, სადაც ყველა ქვესისტემა ერთმანეთს ხვდება (იხ. დიაგრამა 2): დოკუმენტის აქტორი, WebSocket კავშირი, სესიის მდგომარეობის მანქანა, პროცესების მართვა, ნაგვის შემგროვებელი, ფაილების შენახვა, უფლებების სისტემა და მეტრიკები.

დოკუმენტის აქტორი. ყველაზე მნიშვნელოვანი დიზაინის გადაწყვეტილება დოკუმენტთან წვდომის ორგანიზებაა. გულუბრყვილო მიდგომა იქნებოდა `Mutex<Document>` - საზიარო ობიექტი საკეტით, რომელსაც ყველა ნაკადი ეპოტინება. ამის ნაცვლად აქტორის ნიმუში გამოვიყენეთ: დოკუმენტს ფლობს ერთადერთი tokio ამოცანა (`DocActor`), რომელიც უსასრულო ციკლში კითხულობს შეტყობინებების არხს (`mpsc`). ნებისმიერი მოქმედება, ლოკალური ჩასმა, დაშორებული ოპერაცია, სნეფშოტის აღება, ფაილში ჩასაწერი ტექსტის წაკითხვა, არხში იგზავნება `DocOp` შეტყობინებად, პასუხი კი ერთჯერადი (`oneshot`) არხით ბრუნდება. ამ მიდგომას რამდენიმე უპირატესობა აქვს: საკეტები და `deadlock`-ის რისკი ქრება, ოპერაციები ბუნებრივად სერიალიზდება (რაც CRDT-ისთვის ისედაც აუცილებელია), ხოლო ახალი ოპერაციის დამატება მხოლოდ ახალი `DocOp` ვარიანტის და მისი დამმუშავებლის დაწერას ნიშნავს. Rust-ის `ownership` სისტემა აქ ჩვენს სასარგებლოდ მუშაობს - დოკუმენტზე მიმართვა აქტორის გარეშე უბრალოდ ვერ „გაიპარება“.

IPC, ბექენდიდან ფრონტენდამდე. როდესაც აქტორი დაშორებულ ცვლილებას აინტეგრირებს, ფრონტენდს Tauri ივენთს უგზავნის: `crdt://remote-change` (პოზიცია და შიგთავსი), `crdt://snapshot-applied` (დოკუმენტი მთლიანად შეიცვალა, მაგ. სესიაში შესვლისას) ან `crdt://document-reset`. საპირისპირო მიმართულებით ფრონტენდი `#[tauri::command]` ბრძანებებს იძახებს: `insert`, `delete`, `replace`, ფაილის ოპერაციები, სესიის მართვა და ა.შ. საინტერესო ნიუანსი: `webview`-სა და ბექენდს შორის შეტყობინებები ასინქრონულია, ამიტომ შესაძლებელია, რომ მომხმარებელმა პოზიციაზე დააჭიროს მაშინ, როცა ბექენდში უკვე ჩამოვიდა (მაგრამ რედაქტორში ჯერ არ ასახულა) სხვისი ცვლილება. ამის გამო ყოველი ლოკალური ბრძანება თან ატარებს `base_seq`-ს, ბოლო დაშორებული ცვლილების ნომერს, რომელიც რედაქტორს ჰქონდა ნანახი ბრძანების გაგზავნისას. აქტორი ინახავს ბოლო დაშორებული ოპერაციების მცირე ჟურნალს და შემოსული პოზიციის მათ მიმართ ტრანსფორმაციას აკეთებს, პატარა, OT-ის სულისკვეთების ხიდი `webview`-სა და ბექენდის რეპლიკას შორის, რომელმაც სინქრონიზაციის რთულად დასაჭერი ბაგები აღმოფხვრა.

პროცესების მართვა. სესიის დაწყებისას `process_coordinator` უშვებს გეითვეის `sidecar`-ს, სტანდარტული გამოსავლიდან კითხულობს `{"port": N}` შეტყობინებას, შემდეგ (საუკეთესო ძალისხმევით) უშვებს `cloudflared`-ს და ლოგებიდან იჭერს გაცემულ საჯარო ბმულს. გეითვეისთან ავტორიზაციისთვის აპლიკაცია ყოველ სესიაზე შემთხვევით `bearer` ტოკენს აგენერირებს და პროცესს გარემოს ცვლადით გადასცემს, ამ ტოკენით ხდება ოთახის შექმნა და სესიის დასრულება (თავად `WebSocket` შეერთება ტოკენს არ ითხოვს, რადგან სტუმრებს მისი ცოდნა არ სჭირდებათ და არც უნდა სჭირდებოდეთ). ცალკე ყურადღება დაეთმო პროცესების სიცოცხლის ციკლს: ფანჯრის დახურვისას აპლიკაცია სინქრონულად შლის ოთახს და კლავს ორივე `sidecar`-ს, რომ მომხმარებელს „ობოლი“ სერვერი არ დარჩეს ფონურად გაშვებული.

ასინქრონულობა და WebSocket მართვა. მთელი ბექენდი `tokio`-ს ასინქრონულ runtime-ზე დგას. `WebSocket` კავშირს სამი დამოუკიდებელი `tokio` ამოცანა ემსახურება: `write_loop` (გამავალი ფრეიმების რიგი), `receive_loop` (შემომავალი ბაიტების მიღება და ტიპის ამოცნობა) და `process_loop` (მიღებული ოპერაციების გადაცემა დოკუმენტის აქტორისთვის და სხვა ქვესისტემებისთვის). ამოცანების ეს გამიჯვნა ორ რამეს გვაძლევს: ნელი ქსელი ვერასდროს ბლოკავს დოკუმენტის

დამუშავებას, ხოლო კავშირის გაწყვეტისას საკმარისია სამივე ამოცანის გაუქმება და მდგომარეობის გაწმენდა ერთ ადგილას (disconnect handler).

სესიის მდგომარეობის მანქანა. სესიის მდგომარეობა მკაცრი სასრული ავტომატია (FSM): Undecided → Starting → Host/Guest → Undecided. ყველა გადასვლა ვალიდირდება და უკანონო გადასვლა შეცდომას აბრუნებს. სესიის წამოწყება მრავალსაფეხურიანი ასინქრონული პროცესია (პროცესების გაშვება, ოთახის შექმნა, WebSocket კავშირი) და ნებისმიერ საფეხურზე შეიძლება ჩავარდეს, ამიტომ Starting მდგომარეობას RAII „მცველი“ (guard) იცავს: თუ ფუნქცია შეცდომით დასრულდა და მცველი ისე განადგურდა, რომ სესია არ დასრულებულა, მდგომარეობა ავტომატურად ბრუნდება Undecided-ზე. ასე გამოვრიცხეთ „გაჭედილი“ შუალედური მდგომარეობები ხელით rollback-ების წერის გარეშე.

უფლებების მართვა. ჩაწერის უფლებების ავტორიტეტი გეითვეია: ის პირველ შემოერთებულს ჰოსტად თვლის, ყოველ კლიენტს ჩაწერის დროშას უნახავს და წაკითხვის რეჟიმში მყოფი კლიენტის ოპერაციებს უბრალოდ აგდებს (კლიენტის კოდის მოდიფიცირებული ვერსიაც რომ შეეცადოს წესის გვერდის ავლას, გეითვეი მის ოპერაციებს არ გაავრცელებს). ჰოსტი უფლების შეცვლას სპეციალური ფრეიმით (0x07) ითხოვს, გეითვეი ამოწმებს, რომ მოთხოვნა ნამდვილად ჰოსტისგან მოდის, ცვლის დროშას და იმავე ფრეიმს ყველას ავტორიტეტულ ექოდ უგზავნის. კლიენტის მხარეს roster მოდული აწარმოებს მონაწილეთა სიას (სახელები, ჰოსტობა, უფლებები), ხოლო სტუმრის საკუთარი უფლება AppRole::Guest-შივე ინახება და insert/delete/replace ბრძანებებს კარიბჭესავით კეტავს. ინტერფეისში ეს Monaco-ს readOnly რეჟიმად აისახება. აქ ერთი ხრიკი დაგვჭირდა: Monaco წაკითხვის რეჟიმში პროგრამულ ცვლილებებსაც ჩუმად აგდებს, ამიტომ დაშორებული ცვლილების ჩასმისას დროშას წამიერად ვხსნით და ისევ ვაბრუნებთ.

ფაილების შენახვა და გახსნა. დოკუმენტი ორ ფორმატში ინახება, გაფართოების მიხედვით: ჩვეულებრივი ტექსტი (ნებისმიერი გაფართოება) ან საკუთარი .pcdoc ფორმატი, PCDC მაგიური ბაიტები, ვერსია და ბინარულად სერიალიზებული სრული CRDT სნეფშოტი, ისტორიითა და იდენტიფიკატორებით. გახსნისას ფორმატს ვცნობთ არა გაფართოებით, არამედ მაგიური ბაიტებით. ჩაწერა ყოველთვის ატომურია, ჯერ დროებით ფაილში, შემდეგ გადარქმევით, რომ შენახვისას შეწყვეტამ

(გათიშულმა დენმა, ჩავარდნილმა პროცესმა) ნახევრად ჩაწერილი ფაილი არ დაგვიტოვოს. ცალკე პრობლემა აღმოჩნდა ხაზის დაბოლოებები: Windows-ის ფაილები CRLF-ს იყენებს, Monaco და CRDT კი LF-ს ელოდება. გახსნისას ტექსტს LF-ზე ვანორმალიზებთ, ვიმასსოვრებთ, იყო თუ არა ფაილი CRLF-იანი, და შენახვისას ისევ ვაბრუნებთ, ასე Windows-ის ფაილი ბაიტ-ბაიტ იდენტური ბრუნდება და კროსპლატფორმულ სესიაშიც პოზიციები აღარ „ცურავს“ (ეს ბაგი რეალურად გვქონდა და ქვემოთ, პრობლემების ნაწილში, ცალკე ვახსენებთ). ბოლოს, აპლიკაცია ბოლო გახსნილი ფაილების სიას (MRU) აწარმოებს, სრულფასოვანი ფაილ-მენეჯერის გარეშე, უბრალოდ სწრაფი წვდომისთვის.

მეტრიკების სისტემა. დიაგნოსტიკისთვის აპლიკაციას ჩაშენებული მეტრიკების პანელი აქვს. გეითვეი ატომურ მთვლელებს აწარმოებს, აქტიური ოთახები და კლიენტები, გადაგზავნილი ფრეიმების რაოდენობა და მოცულობა, სნეფშოტ-სინქრონიზაციების წარმატება/ჩავარდნა, ნელი კლიენტების გათიშვები, და მათ HTTP ენდპოინტზე აქვეყნებს. ჰოსტის აპლიკაციაში ცალკე tokio ამოცანა (process_metrics_aggregator) პერიოდულად კითხულობს გეითვეის ამ ენდპოინტს და cloudflared-ის საკუთარ მეტრიკებს, აერთიანებს მათ და ივენთით ფრონტენდს აწვდის, სადაც მომხმარებელი მათ პანელში ხედავს. ეს სესიის გამართვისას ძალიან გამოგვადგა, მაშინვე ჩანს, გადის თუ არა ტრაფიკი და ვინ არის შეერთებული.

ფრონტენდი

ფრონტენდი React + TypeScript აპლიკაციაა (აწყობილი Vite-ით), რომლის ცენტრშიც Monaco რედაქტორი დგას. მისი მთავარი ამოცანა ორმხრივი ნაკადის გამართული მუშაობაა: ლოკალური კლავიშები ბექენდისკენ, დაშორებული ცვლილებები, რედაქტორისკენ.

ლოკალური ცვლილებების გაგზავნა. Monaco-ს ყოველი შინაარსობრივი ცვლილება (onDidChangeModelContent) ითარგმნება IPC ბრძანებად: insert, delete ან replace, პოზიციით და შიგთავსით. იმისთვის, რომ სწრაფი ბეჭდვისას ბრძანებები ერთმანეთს არ გადაუსწრონ, ისინი ერთ ჯაჭვზე ეწყობა (opQueue, ყოველი შემდეგი ბრძანება წინას დასრულებას ელოდება). თითო ბრძანებას თან მიჰყვება baseSeq, ბოლო დაშორებული ცვლილების ნომერი, რომელიც რედაქტორს გაგზავნის მომენტში ჰქონდა ასახული, რომლითაც ბექენდი პოზიციას საჭიროებისას გადათვლის (იხ. წინა თავი).

დაშორებული ცვლილებების მიღება. ბექენდის `crdt://remote-change` ივენთს ისმენს `remoteChangeListener`, რომელიც ცვლილებას Monaco-ში პროგრამულად ჩასვამს. აქ ორი ხაფანგი გვხვდება: ჯერ ერთი, პროგრამული ჩასმაც `onDidChangeModelContent`-ს ააქტიურებს, რაც ცვლილებას უკან, ბექენდისკენ გააბრუნებდა და უსასრულო ციკლს გააჩენდა, ამას ვკეტავთ `isApplyingRemote` დროშით, რომელიც ჩასმის განმავლობაში ლოკალურ დამმუშავებელს თიშავს. მეორე, წაკითხვის რეჟიმში მყოფ რედაქტორში Monaco პროგრამულ ცვლილებებსაც უგულებელყოფს, ამიტომ დაშორებული ჩასმისას `readOnly` დროშა წამიერად იხსნება. სნეფშოტის მიღებისას (მაგ. სესიაში შესვლისას) ცალკე მსმენელი მთელ დოკუმენტს ანაცვლებს.

ინტერფეისის აგებულება. ვიზუალურად აპლიკაცია VS Code-ის სტილის გარსია: მარცხნივ ვიწრო ხატულების ზოლი (side rail), რომელიც გვერდით პანელს ხსნის სამი სექციით - ფაილები (გახსნა/შენახვა/ბოლო ფაილები), თანამშრომლობა (სესიის წამოწყება/შეერთება/დატოვება) და პერსონალური პარამეტრები (სახელი, თემა, ფონტის ზომა). ზედა ზოლში სესიის სტატუსი, მონაწილეთა ავატარები (თითოეულს დეტერმინისტულად გამოთვლილი ფერი აქვს), მოსაწვევი ბმულების popover და მენიუა. მონაწილეთა პანელი ჰოსტს თითო სტუმართან ჩაწერის უფლების ჩამრთველს აძლევს. ინტერფეისის მდგომარეობა React ჰუკებშია ორგანიზებული (`useRoomState`, `useSessionEvents`, `useWritePermission`, `useTheme` და ა.შ.), რომლებიც ბექენდის ივენთებს უსმენენ და კომპონენტებს უბრალო props-ებით კვებავენ. ერთი ტექნიკური წვრილმანი: კლიენტების იდენტიფიკატორები 64-ბიტიანი მთელი რიცხვებია, JavaScript-ის რიცხვები კი 2^{53} -ზე დიდ მთელეებს კარგავენ - ამიტომ იდენტიფიკატორები ფრონტენდზე ყოველთვის სტრიქონებად გადადის.

ფრონტენდის ვიზუალური დიზაინის იტერაციებში (ფერთა პალიტრა, კომპონენტების განლაგება, თემები) დამხმარე ინსტრუმენტად ვიყენებდით Claude-ს [9]; საბოლოო დიზაინის გადაწყვეტილებები და მათი იმპლემენტაცია გუნდის წევრებისაა.

ფრონტენდს აქვს საკუთარი ტესტებიც (vitest): სინქრონიზაციის კრიტიკული დამხმარე ლოგიკა, შებრუნებული რედაქტირებების გამოთვლა და `baseSeq`-ის აღრიცხვა, ავტომატურად მოწმდება, ხოლო ტიპიზაცია და ლინტერი CI-ში ეშვება.

გეითვეი (Go სარელეო სერვერი)

გეითვეი განზრახ „სულელი“ სერვერია (dumb relay): ის დოკუმენტის შესახებ არაფერს ინახავს, არც სნეფშოტს, არც ოპერაციების ისტორიას, და CRDT ფრეიმების შიგთავსს არც კი შიფრავს. მისი საქმეა ბაიტების სწრაფად და სამართლიანად გადაგზავნა. ეს გადაწყვეტილება პირდაპირ გამომდინარეობს პროექტის ფილოსოფიიდან: სიმართლის წყარო მონაწილეების (და პირველ რიგში ჰოსტის) რეპლიკებია და სერვერს დოკუმენტზე წვდომა არ სჭირდება.

მისი ძირითადი ცნებებია:

- **Hub**, ოთახების რეესტრი: ქმნის ოთახებს (POST /rooms), პოულობს მათ იდენტიფიკატორით და შლის სესიის ბოლოს;
- **Room**, ერთი სესიის სივრცე: წევრების სია და გადაგზავნის ლოგიკა. მიღებული ფრეიმი ეგზავნება ოთახის ყველა წევრს გამგზავნის გარდა;
- **Client**, ერთი WebSocket შეერთება: როლი (პირველი შემოერთებული ჰოსტია), ჩაწერის უფლების ატომური დროშა და გამავალი ფრეიმების ბუფერი.

მიუხედავად „სისულელისა“, გეითვეის რამდენიმე აქტიური მოვალეობა მაინც აქვს. ახალი წევრის შემოსვლისას ის ჰოსტს სნეფშოტის მოთხოვნის საკონტროლო ფრეიმს უგზავნის და ჰოსტის პასუხს მხოლოდ ახალ წევრს გადასცემს, ასე გვიან შემოსული სტუმარი წამებში იღებს დოკუმენტის მიმდინარე მდგომარეობას, ისე, რომ სერვერს დოკუმენტის შენახვა არ დასჭირვებია. წევრის ყოველ შემოსვლა-გასვლაზე ოთახი ყველას უგზავნის წევრობის ფრეიმს (ამას ნაგვის შემგროვებელი იყენებს) და ახალ წევრს, არსებული წევრების სრულ სიას სახელებითა და უფლებებით. უფლებების ფრეიმებზე ის ავტორიტეტულ არბიტრს თამაშობს: მოთხოვნას მხოლოდ ჰოსტისგან იღებს, ჰოსტის საკუთარი უფლების შეცვლას კრძალავს, ხოლო წაკითხვის რეჟიმში მყოფი წევრის ოპერაციებს ჩუმად აგდებს.

უსაფრთხოებისა და მდგრადობის მხრივ: ყველა HTTP ენდპოინტი (გარდა /health და /ws) მოითხოვს bearer ტოკენს, რომელიც მხოლოდ ჰოსტის აპლიკაციამ იცის; WebSocket შეერთებების სიხშირე შეზღუდულია (ნაგულისხმევად 5 შეერთება წუთში ერთი მისამართიდან, კონფიგურირებადი); ხოლო თუ რომელიმე კლიენტი ფრეიმებს ვერ ასწრებს მიღებას (ნელი ქსელი, გაჭედილი პროცესი), მისი ბუფერი ივსება და გეითვეი მას გათიშავს, რომ დანარჩენი ოთახი არ შეაფერხოს, ეს ე.წ. slow client problem-ის სტანდარტული გადაწყვეტაა. ამას ემატება

ზემოთ ნახსენები მეტრიკების რეესტრი, რომელსაც ჰოსტის აპლიკაცია პერიოდულად კითხულობს.

გეითვეი მთლიანად უმდგომარეო (stateless) და ერთსესიიანია: ის ჰოსტის აპლიკაციასთან ერთად იბადება, ერთ ოთახს ემსახურება და სესიის დასრულებისას (POST /end-session ან ჰოსტის ფანჯრის დახურვისას) წევრებს სესიის დასრულების ფრეიმს უგზავნის და ითიშება. Go ამ ამოცანას ბუნებრივად მოერგო: goroutine-ები და არხები ზუსტად ის პრიმიტივებია, რაც „თითო კლიენტზე ორი მარყუჟი, ოთახზე ერთი გამანაწილებელი“ ტიპის არქიტექტურას სჭირდება, ხოლო ერთი სტატიკური ბინარი sidecar-ად გასაშვებად იდეალურია.

სისტემის ოპტიმიზაცია

პროექტის მეორე ნახევარში ორი დიდი ოპტიმიზაცია განვახორციელეთ: პოზიციური B+ ინდექსი და ნაგვის შემგროვებელი სისტემა. ორივე იმ პრობლემებს პასუხობს, რომლებიც ყველა სერიოზულ CRDT იმპლემენტაციას აწუხებს და რომელთა გადაწყვეტასაც Yjs [4] საკუთარ დიზაინში დიდ ადგილს უთმობს.

პოზიციური B+ ინდექსი რიცხვებში. ინდექსის აგებულება CRDT-ის თავში აღვწერეთ; აქ მის ეფექტზე ვისაუბრებთ. ოპტიმიზაციამდე ყოველი კლავიში დოკუმენტის დაკავშირებული სიის თავიდან ჩამოვლას მოითხოვდა, $O(n)$ ორივე მიმართულებით (პოზიციიდან ბლოკამდე და ბლოკიდან პოზიციამდე). მცირე ფაილებზე ეს შეუმჩნეველია, მაგრამ ბლოკების რაოდენობის ზრდასთან ერთად ღირებულება წრფივად იზრდება და ეს ღირებულება ყოველ კლავიშზე და ყოველ დაშორებულ ოპერაციაზე გადაიხდება. ჩვენი გაზომვებით (debug ბილდი, სადაც ხის განშტოების ფაქტორი განზრახ მცირეა):

ბლოკების რაოდენობა	სიის ჩამოვლა ($O(n)$)	B+ ხე ($O(\log n)$)	აჩქარება
~200	~127 μs	~14 μs	9×
1 000	~620 μs	~16 μs	~ 39×
10 000	~6 200 μs	~18 μs	~ 340×

ცხრილი 1. პოზიციის ძებნის დრო ინდექსამდე და ინდექსის შემდეგ

საკვანძო დაკვირვება ისაა, რომ ხის სვეტი თითქმის მუდმივია, ლოგარითმი პრაქტიკაში „ბრტყელია“. Release ბილდში (განშტოება 64/32, სრული ოპტიმიზაციები) თითო ძებნა 1 μs -ზე ნაკლებია დოკუმენტის ზომის მიუხედავად, ანუ რედაქტორის კადრის ბიუჯეტს ინდექსი

ვერასდროს ემუქრება. ეს ზუსტად ის მიზეზია, რის გამოც მოწიფული CRDT იმპლემენტაციები (Yjs, diamond-types [5]) მსგავს ინდექსებს აუცილებლობად თვლიან.

ნაგვის შემგროვებელი, რატომ არის საჭირო. როგორც აღვნიშნეთ, CRDT-ში წაშლა მხოლოდ მონიშვნაა და წაშლილი ბლოკები tombstone-ებად რჩება. აქტიური რედაქტირებისას ეს სწრაფად გროვდება: დიდხანს მიმდინარე სესიაში დოკუმენტის მეხსიერების უმეტესი ნაწილი შესაძლოა დიდი ხნის წაშლილმა ტექსტმა დაიკავოს, რომელიც სნეფშოტებთან ერთად ქსელშიც მოგზაურობს. tombstone-ის უბრალოდ წაშლა კი საშიშია: თუ რომელიმე მონაწილემ ის ჯერ არ ნახა, ან თუ დაგვიანებული ოპერაცია მას origin-ად ეყრდნობა, ნაადრევი წაშლა რეპლიკების უხმო დაშორებას (divergence) გამოიწვევს. სწორედ ამიტომ ეთმობა Yjs-ში ნაგვის შემგროვებას ცალკე, საკმაოდ ფაქიზი მექანიზმი.

ჩვენი მიდგომის თავისებურება. აქ ვისარგებლეთ ჩვენი არქიტექტურის ერთი თვისებით, რომელიც „სუფთა“ peer-to-peer სისტემებს არ აქვთ: ჩვენს სესიას ჰყავს გამორჩეული კვანძი - ჰოსტი, რომელიც ყოველთვის ონლაინაა (სესია მის გარეშე ვერ არსებობს) და რომელსაც გეითვეი წევრობის ზუსტ სურათს აწვდის. ეს ჰოსტს ბუნებრივ კოორდინატორად აქცევს, ისე, რომ დოკუმენტის მონაცემები მაინც სრულად დეცენტრალიზებული რჩება. სქემა ასე მუშაობს:

1. ყოველი სტუმარი პერიოდულად (დებანსით, რომ ქსელი არ გადაიტვირთოს) აქვეყნებს თავის მდგომარეობის ვექტორს, „აი, რამდენი მაქვს ნანახი თითოეული ავტორისგან“;
2. ჰოსტი აგროვებს ყველა წევრის ვექტორს (გეითვეის წევრობის ფრეიმები ეუბნება, ვინ არის ამჟამად ოთახში) და ითვლის მათ მინიმუმს - „იატაკს“: ოპერაციების იმ ზღვარს, რომელიც ყველას გარანტირებულად აქვს ნანახი;
3. ამ იატაკს ჰოსტი კვეთს საკუთარ DeleteSet-თან და იღებს „დადასტურებული“ წაშლების სიმრავლეს, წაშლებს, რომლებიც ყველამ ნახა;
4. დადასტურებული დიაპაზონები gc-commit ფრეიმად ეგზავნება ყველა მონაწილეს (და ჰოსტისვე დოკუმენტს), რომლებიც მას ერთნაირად ასრულებენ.

gc-commit-ის შესრულება განზრახ კონსერვატიულია: ბლოკები სტრუქტურულად არ იშლება, მხოლოდ მათი ტექსტური შიგთავსი იცლება

და სნეფშოტისთვის განკუთვნილი წაშლის სიმრავლეები იკვეცება. ბლოკის იდენტიფიკატორი და ადგილი სიაში უცვლელი რჩება, ამიტომ დაგვიანებული ოპერაცია, რომელიც ამ ბლოკს origin-ად იყენებს, ისევ ზუსტად ინტეგრირდება - სწორედ ეს ხდის ჩვენს ნაგვის შეგროვებას კონვერგენციისთვის უსაფრთხოს. სტრუქტურული წაშლა შემდეგი ნაბიჯი იქნებოდა, მაგრამ ის ინტეგრაციის ალგორითმის ინვარიანტებს არღვევს (გაყოფის მიმართ ინვარიანტულობა დაიკარგება) და მეხსიერების მოგება რისკად არ ღირს, მთავარი მოგება (ტექსტის გათავისუფლება და სნეფშოტების შემცირება) ისედაც მიიღწევა.

რამდენადაც ჩვენთვის ცნობილია, ეს ჰოსტ-ავტორიტეტული, მდგომარეობის-ვექტორის-იატაკზე დაფუძნებული სქემა განსხვავდება გავრცელებული CRDT იმპლემენტაციების მიდგომებისგან, რომლებიც ან სრულად სიმეტრიულ პროტოკოლებს იყენებენ, ან ნაგვის შეგროვებას საერთოდ დოკუმენტის „შენახვის“ მომენტამდე დებენ. ჩვენს შემთხვევაში ჰოსტის გამორჩეულობა ისედაც სისტემის ნაწილი იყო, მისი გამოყენება ნაგვის შეგროვების გასამარტივებლად არქიტექტურის ბუნებრივი გაგრძელება აღმოჩნდა.

პრობლემები, რომლებსაც გზაში გადავაწყდით. გაიდლაინის შესაბამისად, აქვე შევაჯამებთ განვითარებისას წამოჭრილ მნიშვნელოვან პრობლემებსა და მათ გადაწყვეტებს:

- **პოზიციების „ცურვა“ webview-სა და ბექენდს შორის** - ასინქრონული IPC-ის გამო ლოკალური ბრძანება შესაძლოა მოძველებულ პოზიციას ატარებდეს. გადაწყვეტა: baseSeq + ბექენდში დაშორებული ოპერაციების მცირე ჟურნალის მიმართ პოზიციის ტრანსფორმაცია.
- **Windows-ის CRLF დესინქრონიზაცია**, CRLF-იანი ფაილის გახსნისას პოზიციები პლატფორმებს შორის აღარ ემთხვეოდა და ხაზის შუაში რედაქტირება ცდებოდა. გადაწყვეტა: LF-ზე ნორმალიზაცია გახსნისას და CRLF-ის აღდგენა შენახვისას.
- **Monaco-ს readOnly ხაფანგი**, წაკითხვის რეჟიმში Monaco პროგრამულ რედაქტირებებსაც აგდებდა და სტუმარი დაშორებულ ცვლილებებს ვეღარ იღებდა. გადაწყვეტა: დროშის წამიერი მოხსნა დაშორებული ცვლილების ჩასმისას.
- **სნეფშოტ-სინქრონიზაციის რბოლა**, თავდაპირველად ჰოსტი სნეფშოტს მხოლოდ შეერთებისას აგზავნიდა და გვიან შემოსული სტუმარი შესაძლოა მოძველებულ მდგომარეობას იღებდა.

გადაწყვეტა: მოთხოვნა- პასუხის სქემა, გეითვეი ყოველ ახალ წევრზე ჰოსტს ახალ სნეფშოტს სთხოვს.

- **დიდი ჩასმების დანაწევრება**, გიგანტური ერთიანი ბლოკები (მაგ. მთელი ფაილის ჩასმა) ინდექსსა და ქსელს ტვირთავდა. გადაწყვეტა: ტექსტის 64-სიმბოლოიან ბლოკებად დაჯრა გახსნისას.
- **ინდექსისა და სიის დაშორება**, B+ ხის დანერგვისას ყველაზე სახიფათო რისკი ორი სტრუქტურის უხმო დაშორება იყო. გადაწყვეტა: debug-ორაკული, რომელიც ყოველი მუტაციის შემდეგ სრულ შედარებას აკეთებს, პლუს ხის შიდა ინვარიანტების ცალკე ვალიდატორი.

მიღებული შედეგი და შეფასება

პროექტის საბოლოო შედეგი არის მომუშავე, კროსპლატფორმული desktop აპლიკაცია, რომელიც Linux-ზე, Windows-სა და macOS-ზე ეშვება. ტიპური სამუშაო ნაკადი ასე გამოიყურება: პირველი გაშვებისას მომხმარებელი სახელს ირჩევს (იმახსოვრება შემდეგი გაშვებებისთვის), შემდეგ ხსნის ან ქმნის დოკუმენტს და გვერდითი პანელიდან იწყებს სესიას. აპლიკაცია რამდენიმე წამში აბრუნებს ორ მოსაწვევ ბმულს, ლოკალური ქსელისა და საჯაროს, რომლებსაც ჰოსტი ერთი დილაკით აზიარებს. სტუმრები საკუთარ აპლიკაციაში ბმულს ჩასვამენ და მაშინვე ხედავენ დოკუმენტის მიმდინარე მდგომარეობას; ყველა შემდგომი ცვლილება ყველასთან რეალურ დროში აისახება. ჰოსტი მონაწილეთა პანელიდან მართავს, ვის შეუძლია რედაქტირება, ხოლო ნამუშევარი ნებისმიერ დროს ინახება როგორც ჩვეულებრივ ტექსტურ ფაილად, ისე .pcdoc ფორმატში (სრული ისტორიით).

[insert screenshot: first-run username prompt]

[insert screenshot: main editor window with side rail and side panel open]

[insert screenshot: hosting flow — invite popover with LAN and public URLs]

[insert screenshot: two clients editing the same document simultaneously]

[insert screenshot: peers panel with per-guest write-permission toggles]

[insert screenshot: file section — open/save/recents; metrics popup]

მიზნების შესრულება. შესავალში ჩამოყალიბებული ოთხივე მიზანი მიღწეულია: (1) სესიის წამოწყება და ბმულით შეერთება მუშაობს ორივე ქსელურ რეჟიმში, ლოკალურ ქსელში ინტერნეტის გარეშე და გლობალურად Cloudflare Tunnel-ით; (2) პარალელური რედაქტირება უკონფლიქტოდ ერწყმის YATA CRDT-ით და მონაწილეები გარანტირებულად ერთსა და იმავე ტექსტს ხედავენ; (3) ინფრასტრუქტურის მართვა სრულად ავტომატიზებულია, მომხმარებელი სერვერს ვერც ამჩნევს; (4) ჰოსტი სესიას სრულად აკონტროლებს უფლებების ინდივიდუალური მართვით.

ხარისხის უზრუნველყოფა. სამივე კომპონენტს საკუთარი ტესტები აქვს: CRDT ბიბლიოთეკაში - ინტეგრაციის, გაყოფის, წაშლისა და ინდექსის ტესტები (მათ შორის კონფლიქტური სცენარები და debug-ორაკული), გეითვეიში, ოთახის, გადაგზავნისა და უფლებების ტესტები, ფრონტენდზე, სინქრონიზაციის დამხმარე ლოგიკის ტესტები. ბინარული პროტოკოლის ორივე მხარეს protocol drift ტესტები იცავს. CI (GitHub Actions) ყოველ ცვლილებაზე უშვებს ფორმატირების, ლინტერების (Rust-ში clippy warnings

შეცდომებად ითვლება), ტესტებისა და უსაფრთხოების შემოწმებებს (cargo audit, npm audit, govulncheck), ხოლო pre-push hook იმავე შემოწმებებს ლოკალურადაც აწესებს.

დარჩენილი შეზღუდვები. პატიოსანი შეფასება მოითხოვს, აღვნიშნოთ ისიც, რაც სისტემას ჯერ არ შეუძლია: სესია ერთ დოკუმენტზეა შემოსაზღვრული (მრავალფაილიანი პროექტების მხარდაჭერა არ არის), მონაწილეთა კურსორები ერთმანეთისთვის ჯერ არ ჩანს, ხოლო ჰოსტის გათიშვა სესიას ასრულებს, რეზერვური ჰოსტის არჩევის მექანიზმი არ გვაქვს. ეს პუნქტები სამომავლო განვითარების ბუნებრივი კანდიდატებია (იხ. დასკვნა).

დასკვნა

Peared-ის შექმნამ ერთ პროექტში მოაქცია ის, რასაც ჩვეულებრივ რამდენიმე სხვადასხვა კურსი ფარავს: განაწილებული სისტემების თეორია (CRDT, მდგომარეობის ვექტორები, კონვერგენციის მტკიცება), მონაცემთა სტრუქტურები (აუგმენტირებული B+ ხე), სისტემური პროგრამირება Rust-ზე, ქსელური პროგრამირება Go-ზე და თანამედროვე დესკტოპ/ვებ ინტერფეისები. საბოლოო პროდუქტი, ერთობლივი რედაქტორი, რომელიც ცენტრალური სერვისის გარეშე მუშაობს როგორც ლოკალურ, ისე გლობალურ ქსელში, თავიდანვე დასახულ მიზანს პასუხობს და რეალურ სცენარში (აუდიტორია არასტაბილური ინტერნეტით) ნამდვილად გამოსადეგია.

ტექნიკური ცოდნის მიღმა პროექტმა რამდენიმე უფრო ზოგადი გაკვეთილი გვასწავლა. პირველი, კორექტულობის ინფრასტრუქტურის ღირებულება: debug- ორაკულმა, protocol drift ტესტებმა და მკაცრმა CI-მ განაწილებული სისტემის ყველაზე საშიში კლასის შეცდომები (რეპლიკების უხმო დაშორება) განვითარების ეტაპზე დაიჭირა, სანამ ისინი „იშვიათ, გაუმეორებელ ბაგებად“ იქცეოდნენ. მეორე, აბსტრაქციის საზღვრების სწორად გავლება: ის, რომ crdt-core არაფერს იცის ქსელზე, გეითვეი არაფერს იცის დოკუმენტზე, ხოლო ფრონტენდი არაფერს იცის CRDT-ზე, სამივე კომპონენტის დამოუკიდებლად ტესტირებას და პარალელურად განვითარებას აძლევდა საშუალებას. მესამე, გუნდური მუშაობის დისციპლინა: სამი ადამიანის ნამუშევრის ყოველდღიური შერწყმა pull request-ებით, კოდის განხილვით და ავტომატური შემოწმებებით თავად პროექტის ისეთივე ნაწილი აღმოჩნდა, როგორც ალგორითმები.

პროექტს განვითარების მკაფიო გზა აქვს. უახლოესი ნაბიჯები ისედაც თვალსაჩინოა: მონაწილეთა კურსორებისა და მონიშვნების ჩვენება (presence), მრავალფაილიანი პროექტების მხარდაჭერა, სესიის შიდა ჩატი. უფრო შორს - ჰოსტის როლის გადაბარება სესიის შეუწყვეტლად (ამის საფუძველი, დოკუმენტის სრული ასლი ყველა მონაწილესთან, უკვე არსებობს), end-to-end შიფრაცია გეითვეის მიმართ ნულოვანი ნდობისთვის და მუდმივი (persistent) ოთახები საკუთარ სერვერზე განთავსების მოყვარულთათვის. ცალკე საინტერესო მიმართულებაა .pcdoc ფორმატის გამოყენება დოკუმენტის ისტორიის დათვალიერებისთვის, ფორმატში სრული რედაქტირების ისტორია ისედაც ინახება და მასზე „დროში გადახვევის“ ფუნქციის აშენება ბუნებრივი გაგრძელება იქნება.

შეჯამებისთვის: დავიწყეთ აუდიტორიაში შემჩნეული პრაქტიკული

პრობლემით და დავამთავრეთ მომუშავე სისტემით, რომლის ასაგებადაც თანამედროვე განაწილებული რედაქტირების თეორიის საკმაოდ ღრმად შესწავლა მოგვიწია. პროექტმა გვაჩვენა, რომ local-first მიდგომა, სადაც მონაცემები მომხმარებელთან რჩება და ინფრასტრუქტურა მის კომპიუტერზევე ცხოვრობს, არა მხოლოდ იდეოლოგიურად მიმზიდველი, არამედ პრაქტიკულად განხორციელებადი ალტერნატივაა ცენტრალიზებული სერვისებისა.

გამოყენებული ლიტერატურა

- [1] P. Nicolaescu, K. Jahns, M. Derntl, and R. Klamma, “Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types,” in Proceedings of the 19th International Conference on Supporting Group Work (GROUP ’16), Sanibel Island, FL, USA, 2016, pp. 39–49.
- [2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free Replicated Data Types,” in Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS ’11), Grenoble, France, 2011, pp. 386–400.
- [3] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” in Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data, Portland, OR, USA, 1989, pp. 399–407.
- [4] K. Jahns, “Yjs Documentation.” [Online]. Available: <https://docs.yjs.dev/>
- [5] J. Gentle, “diamond-types: The world’s fastest CRDT.” [Online]. Available: <https://github.com/josephg/diamond-types>
- [6] Tauri Working Group, “Tauri 2.0 Documentation.” [Online]. Available: <https://v2.tauri.app/>
- [7] Microsoft, “Monaco Editor.” [Online]. Available: <https://microsoft.github.io/monaco-editor/>
- [8] Cloudflare, Inc., “Cloudflare Tunnel Documentation.” [Online]. Available: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- [9] Anthropic, “Claude.” [Online]. Available: <https://claude.com/>
(გამოყენებულია ფრონტენდის ვიზუალური დიზაინის იტერაციებში დამხმარე ინსტრუმენტად)

დანართები

- პროექტის GitHub ბმული: <https://github.com/CarlTeapot/Peared>